JobPilot: 8-Week Compressed Replan

Phase 1 — Learning + Build Plan for a Backend Engineer with Zero AI Knowledge

Pranjal Yadav

Senior Member of Technical Staff, Oracle

Bengaluru, India

01 April 2026 — Version 1.0

Abstract

This document is a merger of three source artefacts — JobPilot_Learning_Build_Plan_v1_0.docx (v1.0), the Backend-to-AI-Builder Roadmap PDF v2.0, and the PAIOS Build Guide v2 — restructured for 20 hours per week over 8 weeks. Warm-up projects are dropped; all hours flow directly to JobPilot. Phase 1 (this document) delivers five agents: Scout, Resume, Application, Tracker, and Outreach — a functional, end-to-end job application pipeline. Phase 2 (Appendix A) covers Email Intelligence, Follow-up, Adaptive Resume Engine, MCP servers, and Prometheus/Grafana observability. The cascading LLM strategy, no-fabrication constraint on resume rewriting, and mandatory Telegram HITL gates are unchanged from v2.0. Total Phase 1 investment: 160 hours across 8 weeks.

Keywords: agentic AI, RAG, LLM, multi-agent orchestration, LangGraph, Model Context Protocol, cascading LLM, LaTeX resume generation, Gmail API, Telegram HITL, job search automation

1. Version Delta — What Changed from v1.0

This document merges three source artefacts. The v2.0 PDF is the authoritative architecture reference. Content present in the v1.0 DOCX but absent from v2.0 — the day-level schedule, risk register, SDE interview map, and daily routine structure — is incorporated throughout. Parameters that diverge from v1.0 are tabulated below.

Parameter	v1.0 DOCX	This Replan
Total weeks	12 weeks at 14h/week	8 weeks at 20h/week (Phase 1)
Total Phase 1 hrs	112h (Stages 1 of 12-wk plan)	160h (8 × 20h)
Warm-up projects	3 projects (from v2.0 PDF)	Dropped — all hours to JobPilot
Phase 1 scope	All 9 agents	5 core agents (Weekends 1–5)
Phase 2 scope	Weekends 9–12	Weekends 6–9 — see Appendix A
Learning structure	Front-loaded weeks 1–3	Tight interleave — learn day = apply day
No-fabrication	Hard constraint	Unchanged — non-negotiable
Telegram HITL	All external actions gated	Unchanged — non-negotiable
LLM routing	5-tier cascade	Unchanged — see Section 4

Table 1. Version delta: v1.0 DOCX versus this replan.

2. The Arithmetic — Scope and Hour Budget

20 hours/week × 8 weeks = 160 hours. The original estimates from each source artefact are:

- v1.0 DOCX: 170–210h total (40–50h learning + 130–160h building). Planned at 14h/week = 12 weeks.
- v2.0 PDF roadmap: 68–82h learning + 130–166h building = 198–248h total. Warm-up projects account for ~30–40h of that learning.
- With warm-ups dropped: ~160–210h. At 20h/week, 160h covers the low end exactly.

The shortfall on the high end (~50h) falls in JobPilot Weekends 6–9: Email Intelligence, Follow-up Agent, Adaptive Resume Engine, MCP servers, and Prometheus/Grafana. These are designated Phase 2. Phase 1 ships a functional pipeline you can use for real job applications from the end of Week 8.

Phase	Scope	Hours	End State
Phase 1 — this plan	Learning Phases 1–5 + JobPilot Weekends 1–5	160h (8wk × 20h)	Scout, Resume, Application, Tracker, Outreach. Functional pipeline.
Phase 2 — Appendix A	Learning Phases 6–7 + JobPilot Weekends 6–9	~80–100h (est.)	Email Intelligence, Follow-up, ARE, MCP, Grafana. Full 9-agent system.

Table 2. Phase breakdown and hour budget.

3. Skill Transfer Map — Oracle Experience to JobPilot

Your Oracle background provides substantial scaffolding for specific learning phases. The table below maps existing competencies to AI concepts, with an explicit statement of how much learning compression each transfer enables. Skills with no backend equivalent require full deliberate study — no amount of engineering experience shortens them.

Existing Skill	AI Concept	Compression
REST API design, Dropwizard	LLM API integration	LLM APIs are HTTP + JSON. Read error codes and rate-limit semantics only (~1h).
Kubernetes, OKE controller loops	LangGraph state machines	Graph state transitions + retry = K8s reconciliation. Study differences, not fundamentals (~2h).
CI/CD pipelines, TeamCity	Agent pipelines + APScheduler	Sequential/parallel task execution with failure handling. Direct analogue (~1h).
Control Plane / Data Plane arch	Agent architecture	Agents are autonomous services with defined I/O contracts. CP/DP mental model applies.
Docker, Helm, Terraform	Docker Compose deployment	Trivial. Study compose-specific syntax only (~1h).
Prometheus (OCI monitoring)	Prometheus + Grafana	Same stack. 2h study not 10h. Phase 2 only.

Existing Skill	AI Concept	Compression
PostgreSQL (PAIOS) + SQLAlchemy	pgvector + ORM	pgvector is a PostgreSQL extension. New concept = vector similarity only.
Security vuln remediation, IDCS	OAuth 2.0 (Gmail API)	Token management, scope, refresh rotation — already practiced.

Table 3. Skill transfer from backend engineering to JobPilot components.

No backend equivalent: Prompt engineering (system prompts, few-shot, output format enforcement), RAG pipelines (embeddings, vector similarity, context window management), multi-agent deliberation (tournament scoring, consensus mechanisms), and LLM-as-judge evaluation patterns. These require full deliberate study.

4. Cascading LLM Strategy

Unchanged from v2.0. Four-tier fallback chain. Claude Sonnet 4.6 is reserved exclusively for quality-critical resume bullet rewriting and the Phase 2 Adaptive Resume Engine Mediator. All other tasks route through free tiers.

Tier	Provider	Cost / 1M tokens	Used For
1 — Primary	DeepSeek V3 / Qwen 2.5	Free (50M tokens/day)	Scoring, extraction, deduplication, outreach, email classification
2 — Fallback	Gemini 2.0 Flash / GPT-4o-mini	Free — $0.60	Overflow when Tier 1 rate-limited
3 — Terminal	Ollama — Llama 3.2 3B (local)	$0	All remote APIs unavailable. GTX 1650 4GB VRAM; Q4_K_M ~2GB fits.
Premium	Claude Sonnet 4.6	$3 in / $15 out	Resume rewriting only. Narrative Agent + Mediator (Phase 2 ARE).

Table 4. Cascading LLM routing strategy. Estimated Phase 1 monthly cost: $1.50–4.

5. Learning Resources

All resources referenced in the 8-week plan with direct URLs. Free unless noted.

Resources:

Phase 1 — LLM Fundamentals

- Karpathy: Intro to LLMs (YouTube, 1h) https://www.youtube.com/watch?v=zjkBMFhNj_g
- Karpathy: Let's build GPT from scratch (YouTube, 2h) https://www.youtube.com/watch?v=kCc8FmEb1nY
- 3Blue1Brown: Neural Networks playlist (YouTube, 3h) https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi
- HuggingFace NLP / LLM Course Ch 1–4 (Free course) https://huggingface.co/learn/nlp-course/chapter1/1
- Google Cloud: Intro to LLMs (Coursera, free) https://www.coursera.org/learn/introduction-to-large-language-models
- OpenAI Tokenizer (tiktoken playground) (Free tool) https://platform.openai.com/tokenizer

Resources:
Phase 2 — Prompt Engineering & LLM APIs

- Anthropic Prompt Engineering Interactive Tutorial (GitHub, free) https://github.com/anthropics/courses/tree/master/prompt_engineering_interactive_tutorial
- Anthropic Prompt Engineering Docs (Official) https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/overview
- OpenAI Prompt Engineering Guide (Official) https://platform.openai.com/docs/guides/prompt-engineering
- DeepLearning.AI: ChatGPT Prompt Engineering (Free, 1h) https://www.deeplearning.ai/short-courses/chatgpt-prompt-engineering-for-developers/
- Anthropic Courses: API Fundamentals + Tool Use (Free) https://github.com/anthropics/courses
- DeepSeek API Docs (Official) https://api-docs.deepseek.com/
- Qwen / DashScope API Docs (Official) https://dashscope.aliyuncs.com/
- Ollama — local LLM runner (Free tool) https://ollama.com/

Resources:
Phase 3 — RAG & Vector Databases

- DeepLearning.AI: RAG Course (Zain Hasan) (Free, ~5h) https://www.deeplearning.ai/short-courses/building-and-evaluating-advanced-rag/
- LangChain Official RAG Tutorial (Free) https://python.langchain.com/docs/tutorials/rag/
- Pinecone Learning Center: What are Embeddings? (Article) https://www.pinecone.io/learn/vector-embeddings/
- ChromaDB Docs (Free) https://docs.trychroma.com/
- pgvector (PostgreSQL extension) (Open source) https://github.com/pgvector/pgvector

Resources:
Phase 4 — Agents, Tool Use & Consensus Patterns

- DeepLearning.AI: AI Agentic Design Patterns (Andrew Ng) (Free, ~6h) https://www.deeplearning.ai/short-courses/ai-agentic-design-patterns-with-autogen/
- LangChain Academy: Introduction to LangGraph (Free) https://academy.langchain.com/courses/intro-to-langgraph
- Harrison Chase: LangGraph walkthrough (YouTube) https://www.youtube.com/watch?v=R8KB-Zcynxc
- CrewAI Documentation (Free) https://docs.crewai.com/

Resources:
Phase 5 — Model Context Protocol

- MCP Official Specification (Free) https://modelcontextprotocol.io/introduction
- MCP Python SDK (pip install mcp) (Open source) https://github.com/modelcontextprotocol/python-sdk
- MCP Reference Servers (GitHub) (Free) https://github.com/modelcontextprotocol/servers
- MCP Quickstart: Build Your First Server (Free) https://modelcontextprotocol.io/quickstart

Resources:
Phase 6–7 — Evaluation, Observability & Deployment (Phase 2)

- LangChain Academy: Observability & Evaluations (Free) https://academy.langchain.com/courses/intro-to-langgraph

- Langfuse (open-source LLM observability) (Free / self-hosted) https://langfuse.com/
- RAGAS (RAG evaluation metrics) (Open source) https://docs.ragas.io/
- Gmail API Python Quickstart (Free, Google) https://developers.google.com/gmail/api/quickstart/python
- Gmail Incremental Sync Guide (historyId) (Free, Google) https://developers.google.com/gmail/api/guides/sync
- Prometheus: Getting Started (Free) https://prometheus.io/docs/prometheus/latest/getting_started/
- prometheus_client Python (Open source) https://github.com/prometheus/client_python
- Grafana: Getting Started (Free) https://grafana.com/docs/grafana/latest/getting-started/

Resources:
External Integrations & Coding Tools

- Playwright Python Async API (Free) https://playwright.dev/python/docs/api/class-playwright
- Apollo.io API Docs (50 free credits/mo, Freemium) https://apolloio.github.io/apollo-api-docs/
- python-telegram-bot v20 Docs (Free) https://python-telegram-bot.readthedocs.io/
- Lever API: POST /postings/{id}/apply (Free ATS) https://hire.lever.co/developer/documentation
- Greenhouse API Docs (Free ATS) https://developers.greenhouse.io/
- TeXLive Install Guide (Free) https://www.tug.org/texlive/
- Overleaf LaTeX CV Templates (Free) https://www.overleaf.com/latex/templates/tagged/cv
- LangSmith Docs (Free tier) https://docs.smith.langchain.com/
- Claude Code (Paid) https://claude.ai/code
- Cursor IDE ($20/month, Paid) https://www.cursor.com/
- Cline — open-source, any LLM provider (Free) https://github.com/cline/cline

6. 8-Week Hours Breakdown

Week	Focus	Learn (h)	Build (h)	Total	Running
1	LLM Fundamentals + Project Scaffolding	8	12	20	20
2	Scout Agent — Job Discovery	5	15	20	40
3	RAG + Resume Agent	7	13	20	60
4	LangGraph + Tracker + Application Agent	6	14	20	80
5	Gmail OAuth + Outreach Agent	4	16	20	100
6	Pipeline Hardening + Integration Tests	2	18	20	120
7	Resume Depth + LaTeX Polish + Outreach v2	2	18	20	140
8	Phase 1 MVP Ship + Docker Compose + Soak Test	1	19	20	160
Total		35	125	160	—

Table 5. Hour distribution across 8 weeks. Learning = 21.9% of total.

Distribution: Learning is front-loaded in Weeks 1–3 (~40–50% per week), tapering to under 10% by Weeks 7–8. By Week 5, build hours dominate. The 35h of learning maps tightly to the first five phases of the v2.0 curriculum: LLM fundamentals, prompt engineering, RAG, agents, and MCP.

7. Week-by-Week Execution Plan

Each week follows the tight-interleave pattern from the v1.0 DOCX: the Learn column and Build column for each day are executed on the same day. No standalone theory days exist. The daily session structure is covered in Section 9.

7.1 Week 1: LLM Fundamentals + Project Scaffolding (20h · Foundation)

Day	Topic	Learn	Build / Deliverable
1	***LLM Concepts***	Karpathy 'Intro to LLMs' (1h). Tokens, context windows, temperature, top-p. youtube	Project repo. docker-compose.yml with PostgreSQL. FastAPI /health endpoint.
2	***Tokenisation + Cost***	tiktoken playground — paste resume, observe token count. DeepSeek free-tier economics. tokenizer	Install Python deps (langchain, openai, httpx). config.yaml parser and .env. First DeepSeek API call.
3	***Prompt Engineering I***	Anthropic docs: system prompts, role prompts, output format enforcement, XML tags. docs	JD scoring system prompt (PRD §5.2). Test against 3 real JDs via DeepSeek API.
4	***Prompt Engineering II***	Few-shot prompting, chain-of-thought, structured JSON output. OpenAI guide	JD keyword extraction prompt (PRD §6.2). Test against 3 real JDs.
5	***Cascading LLM***	DeepSeek + Qwen API docs. Rate limits, error codes, retry semantics. Ollama install. DeepSeek API	CascadingLLM class (PRD §4.2). 4-tier chain: DeepSeek → Qwen → Gemini Flash → Ollama.
6	***Telegram Bot***	python-telegram-bot v20: handlers, inline keyboards, callback queries. docs	Telegram bot live. send_job_notification() with Apply / Skip / Details inline buttons.
7	***Integration Day***	Review and refactor all Week 1 code. Fill any LLM fundamentals gaps.	Wire CascadingLLM + scoring prompt + Telegram. Manual test: paste JD → scored notification in Telegram.

Week 1 deliverable: Repo live. FastAPI /health responding. CascadingLLM class functional with 4-tier fallback. Telegram bot sends job notifications with inline approval buttons.

7.2 Week 2: Scout Agent — Job Discovery (20h · JobPilot Weekend 1)

Day	Topic	Learn	Build / Deliverable
1	***Web Scraping***	httpx async basics. BeautifulSoup DOM parsing. Rate limiting patterns. httpx	Lever generic scraper: {company}.lever.co/api/postings JSON. Parse title, location, description.

Day	Topic	Learn	Build / Deliverable
2	***ATS Scrapers***	Greenhouse API patterns. feedparser for RSS/Atom. Greenhouse API	Greenhouse scraper. RSS auto-detection probe. feedparser integration. 3 ATS scrapers done.
3	***Title Matching***	Fuzzy string matching: Levenshtein, token overlap. rapidfuzz. RapidFuzz	Title pre-filter (PRD §5.2 Stage 1). Hardcoded title list + fuzzy match. Threshold tuning.
4	***Deduplication***	SHA-256 hashing for dedup. PostgreSQL O(1) lookup index. seen_postings table schema.	Dedup pipeline (PRD §5.2 Stage 4). PostgreSQL seen_postings. Geographic filter.
5	***Scout Pipeline***	APScheduler tiered frequency pattern. APScheduler	Wire full Scout Agent. APScheduler: 10min tier-1, 4h tier-2, 24h tier-3.
6	***LinkedIn + Naukri***	LinkedIn scraping constraints (30/day). Naukri patterns. Playwright	LinkedIn scraper. Naukri scraper. Both integrated into Scout.
7	***Scout Tests***	Edge cases: empty JDs, non-English, malformed HTML, ATS form detection.	test_scout.py. Run against 10 real career pages. Fix failures. Scout Agent v1 complete.

Week 2 deliverable: Scout Agent polls LinkedIn, Naukri, and 200 career pages. Relevant postings (≥7 relevance score) trigger Telegram notification with Apply/Skip. JobPilot Weekend 1 complete.

7.3 Week 3: RAG + Resume Agent (20h · JobPilot Weekend 2)

Day	Topic	Learn	Build / Deliverable
1	***Embeddings***	What are embeddings. Cosine similarity. Pinecone article	Install pgvector on PostgreSQL. Create embeddings table. Test with sentence-transformers.
2	***RAG Pipeline***	LangChain RAG: loading, splitting, embedding, retrieval. LangChain RAG	Embed resume + 5 real JDs. Query by cosine similarity. Compare to keyword matching.
3	***Hybrid Scoring***	Hybrid: LLM + embedding similarity. Context window budgeting. ChromaDB	Add pgvector pre-scoring to Scout Agent. Compare hybrid vs. LLM-only on 20 real postings.
4	***LaTeX Parsing***	Your .tex template: \\section, \\resumeSubheading, \\resumeItem. Regex patterns. Overleaf	LaTeX parser: sections, bullets, skills → structured intermediate representation.
5	***Bullet Rewriting***	Resume rewriting prompt constraints (PRD §6.2). Harvard action verbs. Anthropic Tool Use	NO-FABRICATION — hard constraint, never relaxed. Bullet rewriting via Claude Sonnet. Test on 3 JD pairings.
6	***LaTeX Compile***	pdflatex CLI. Error parsing. Auto-fix for &, %, $. TeXLive	compile_latex() function. Automated error recovery loop. Test compilation end-to-end.
7	***ATS Scoring***	Keyword match rate. Bullet reordering. Skills section reordering. Resume Agent integration.	ATS score estimation. Reordering logic. Resume Agent end-to-end test. Weekend 2 complete.

Week 3 deliverable: Resume Agent parses LaTeX, rewrites bullets with Harvard action verbs under strict no-fabrication rules, compiles PDF, and estimates ATS score. JobPilot Weekend 2 complete.

7.4 Week 4: LangGraph + Tracker + Application Agent (20h · Weekends 3–4)

Day	Topic	Learn	Build / Deliverable
1	***LangGraph I***	StateGraph, nodes, edges, conditional routing. LangChain Academy	Minimal LangGraph: 2 nodes (Scout → Resume) with shared state. First graph executes.
2	***LangGraph II — HITL***	Checkpoints, interrupt_before, human approval nodes, state persistence. HITL tutorial	Telegram HITL checkpoint between Scout and Resume. Mandatory approval before resume tailoring.
3	***LangGraph III***	Parallel execution, fan-out/fan-in. Error handling and retry nodes.	Scout → [HITL] → Resume → [HITL] → Application (stub). Full state schema defined.
4	***Tracker Agent***	openpyxl: workbooks, sheets, conditional formatting. openpyxl	Tracker Agent (PRD §9). 25-column schema. Auto-populate from pipeline state. Weekend 3 done.
5	***Tracker Polish***	Conditional formatting: red/yellow/green status rows. APScheduler daily summary.	Status row formatting. Daily 21:00 IST Telegram summary via APScheduler.
6	***Application Agent***	Lever POST /postings/{id}/apply. Gmail messages.send. Lever API	Telegram approval gate mandatory before any submission. Lever + email + manual fallback.
7	***Integration***	End-to-end: Scout → Resume → Application → Tracker. Error handling. Idempotency checks.	Pipeline test against 5 real postings. Fix integration bugs. Weekend 4 complete.

Week 4 deliverable: Scout → Resume → Application → Tracker pipeline operational. Telegram approval gates before resume tailoring and before every submission. Tracker auto-populates on every state transition. JobPilot Weekends 3 and 4 complete.

7.5 Week 5: Gmail OAuth + Outreach Agent (20h · JobPilot Weekend 5)

Day	Topic	Learn	Build / Deliverable
1	***Gmail OAuth***	Google Cloud Console: project, Gmail API, OAuth credentials. OAuth 2.0 flow. Quickstart	setup_gmail_oauth.py. Obtain refresh token. Test gmail.readonly scope. Auth complete.
2	***Gmail Send***	Gmail API: messages.send(), MIME construction, PDF attachment pattern.	Email submission via Gmail API. Resume PDF attached. Delivery confirmation.
3	***Playwright***	Playwright Python async: browser launch, selectors, form filling. Playwright docs	Playwright install. LinkedIn Easy Apply prototype. 30/day rate limit enforced.

Day	Topic	Learn	Build / Deliverable
4	***Form Auto-fill***	applicant_profile.yaml population. ATS form field mapping. Greenhouse forms	Form auto-population (PRD §7.3). Test against Lever and Greenhouse forms.
5	***Confirmation Gate***	Confirmation gate flow (PRD §7.2). Multi-step Telegram approval patterns.	Telegram Confirm / Cancel / View Resume buttons. Mandatory gate wired into LangGraph.
6	***Outreach Prep***	Apollo.io API: search_people, 50 free credits/month. Apollo.io API	Apollo.io account. find_recruiters_apollo() (PRD §8.1). Test with 3 target companies.
7	***Outreach Agent***	Dual-variant messages (PRD §8.2). Rate limits (PRD §8.3). All messages Telegram-gated.	Outreach Agent: 2 variants per recruiter. Telegram selection UI. Weekend 5 complete.

Week 5 deliverable: Gmail OAuth authenticated. Application Agent submits via Lever API / email / manual fallback with mandatory Telegram confirmation. Outreach Agent generates dual-variant messages with Telegram selection. JobPilot Weekend 5 complete. All Phase 1 core agents built.

7.6 Week 6: Pipeline Hardening + Integration Tests (20h · Cross-agent)

Day	Topic	Learn	Build / Deliverable
1	***Error Hardening***	Retry semantics per agent. Dead-letter queue pattern. Circuit breaker for LLM calls.	Retry + exponential backoff in CascadingLLM. Dead-letter Telegram alert on agent failure.
2	***State Persistence***	LangGraph checkpointing to PostgreSQL. Recovery from partial pipeline runs.	PostgreSQL-backed LangGraph checkpointer. Verify pipeline resumes after crash at any node.
3	***Idempotency***	Idempotent submission keys. Duplicate detection across ATS and email paths.	Idempotency keys for all Application Agent submissions. Dedup across channels.
4	***Rate Limit Mgmt***	Per-provider rate limit tracking. Tier cascade trigger logic. Token budget monitoring.	Rate limit tracker across 4 LLM tiers. Auto-cascade on 429. Token budget logging.
5	***Telegram UX***	Callback query handling. Message edit vs. new message. Approval timeout handling.	Refactor all Telegram handlers. 24h approval timeout → auto-skip with notification.
6	***Integration Tests I***	Test pyramid: unit → integration → end-to-end design.	test_cascade_llm.py, test_scout.py, test_resume_agent.py. All unit tests pass.
7	***Integration Tests II***	End-to-end pipeline test. 10 synthetic postings.	Full pipeline: Scout → Resume → Application → Tracker → Outreach for 10 postings.

Week 6 deliverable: All five agents integrated and hardened. Retry, idempotency, and state persistence across restarts. Full pipeline integration tests passing.

7.7 Week 7: Resume Depth + LaTeX Polish + Outreach v2 (20h · Quality layer)

Day	Topic	Learn	Build / Deliverable
1	***Prompt Refinement***	Structured output via XML tags. Few-shot resume examples. Constraint chaining. Anthropic docs	Resume rewriting prompt v2. 6 explicit constraints. No-fabrication guardrail test on 10 JDs.
2	***Harvard Verbs YAML***	Harvard action verb taxonomy. Verb-to-role-type mapping. Contextual selection logic.	harvard_verbs.yaml: 200+ verbs by function (delivered, engineered, architected). Verb injector.
3	***ATS Score v2***	Keyword coverage metrics. Bullet reordering by JD term frequency.	ATS score v2: keyword coverage (40%) + bullet relevance (40%) + format compliance (20%).
4	***LaTeX Error Recovery***	pdflatex error taxonomy. Auto-escape rules for &, %, $, _, ^.	Automated LaTeX error recovery: parse pdflatex stderr, apply escape rules, retry 3×.
5	***Resume Versioning***	Git-based versioning mental model. Symlink pattern for current version.	resume_versions/ directory. Symlinks. changelog.md. /revert Telegram command.
6	***Batch Test***	Benchmark Resume Agent across 20 JD types. Measure ATS score distribution.	Batch test: 20 JDs × resume rewriting. Record ATS scores, latency, LLM tier used.
7	***Outreach Agent v2***	Message quality review. Personalisation depth. Apollo.io Playwright fallback.	Outreach Agent v2: Playwright fallback on Apollo exhaustion. Message quality scoring gate.

Week 7 deliverable: Resume Agent producing high-quality ATS-optimised PDFs. LaTeX error auto-recovery. Resume versioning with Telegram /revert. Outreach Agent with Playwright fallback.

7.8 Week 8: Phase 1 MVP Ship + Docker Compose (20h · Phase 1 complete)

Day	Topic	Learn	Build / Deliverable
1	***Docker Compose***	Health checks, restart policies, volumes, log rotation. Docker Compose docs	Final docker-compose.yml: app + PostgreSQL + pgvector. Health checks on all services.
2	***Config Hardening***	Env var validation. Secrets management. .env.example for README.	Startup config validator: fail fast on missing vars. .env.example committed to repo.
3	***Testing I***	Integration test design: agent pairs, pipeline, error paths.	test_resume.py, test_cascade_llm.py, test_scout_dedup.py, test_application_agent.py.
4	***Load Test***	50 concurrent synthetic postings. CascadingLLM under rate-limit pressure.	Load test: 50 postings. Verify no data loss, correct dedup, no LLM budget overrun.
5	***Soak Test***	24-hour unattended run. Monitor PostgreSQL connections, memory, LLM token consumption.	24h soak test. Review logs. Fix any memory leaks or connection pool exhaustion.

Day	Topic	Learn	Build / Deliverable
6	***README + Docs***	README structure: 1-para summary, ASCII architecture diagram, quickstart, cost table.	README.md: architecture, setup, cost reference ($1.50–4/month). Troubleshooting guide.
7	***Phase 1 Ship***	Final gap analysis against PRD requirements.	Tag v1.0-phase1. Update changelog. JobPilot is live — begin submitting applications.

Phase 1 complete: Scout, Resume, Application, Tracker, and Outreach agents operational. Docker Compose deployment with health checks. Telegram HITL on all consequential actions. No-fabrication enforced on all resume rewriting. Estimated monthly operating cost: $1.50–4.

8. Phase 1 Agent Summary

#	Agent	Status (End Wk 8)	Key Integration	HITL Gate
1	Scout	Complete	Lever, Greenhouse, Naukri, LinkedIn, RSS, APScheduler	Apply/Skip notification
2	Resume	Complete	pdflatex, Harvard verbs, pgvector, Claude Sonnet 4.6	Tailored resume approval
3	Application	Complete	Lever/Greenhouse API, Gmail API, Playwright fallback	Confirm before submit
4	Tracker	Complete	openpyxl, PostgreSQL, APScheduler 21:00 IST daily	None — automated
5	Outreach	Complete	Apollo.io, Playwright fallback, LangGraph graph	Message selection per recruiter
6	Email Intel	Phase 2	Gmail API, incremental sync, 2-stage classifier	Manual verify <0.85 confidence
7	Follow-up	Phase 2	SMTP, Telegram approval, APScheduler 10:00 IST	Approve/Edit/Skip/Delay
8	ARE	Phase 2	LangGraph 4-agent tournament, git versioning	Telegram /revert, /pin
9	Analytics	Phase 2	prometheus_client, Grafana 6-row dashboard	None — automated

Table 6. All nine JobPilot agents with Phase 1 / Phase 2 status.

9. Daily Session Structure

Source: v1.0 DOCX Section 7, adapted for 2h 50min sessions (20h ÷ 7 days). The learn/build split is not rigid. For topics covered by existing Oracle experience, compress the learn block to 10 minutes and extend build time accordingly.

Block	Duration	Activity
Orientation	5 min	Open this document. Read today's row. Identify Learn and Build tasks explicitly.

Block	Duration	Activity
Learn Block	35–55 min	Read docs or watch the video. Stop when the concept is understood, not when time expires. For topics your Oracle background covers directly, compress to 10 min.
Build Block	120–145 min	Write the code in the Build column. Test it. Commit to git.
Close	5 min	git commit with week/day reference (e.g. W3D4: LaTeX parser). Note blockers for the next session.

Table 7. Recommended daily session structure.

10. Risk Register

Source: v1.0 DOCX Section 6, updated for the 20h/week, 8-week schedule.

Risk	Probability	Mitigation
LangGraph steeper than estimated	High	Week 4 has 3 dedicated days. If stuck by Day 3: fall back to plain Python orchestrator (while loop + dispatch) for the MVP. Migrate to LangGraph in Phase 2.
LinkedIn scraping triggers restriction	Medium	Apollo.io is the primary recruiter source. LinkedIn scraping capped at 30/day with Playwright. Use a throwaway dev account. Never automate InMail.
20h/week schedule slippage	High	Bank hours from lighter days. A 3h Saturday session covers a missed weekday evening. Weeks 1–3 are highest-risk — front-load effort before novel concepts become blockers.
Free-tier LLM quality insufficient for rewriting	Medium	Claude Sonnet 4.6 is designated for resume rewriting — not a free-tier task. Budget $1.50–3/month. Non-negotiable for quality.
Email misclassifies interview invite as rejection (Phase 2)	Low	0.85 confidence threshold routes ambiguous classifications to Telegram for manual review. No auto-action below threshold.
ARE degrades resume quality (Phase 2)	Medium	Performance floor: <10% acceptance rate for 3 consecutive 12h cycles triggers auto-revert. /pin command freezes a known-good version.

Table 8. Risk register with probability and mitigation.

11. SDE Interview Competence Map

Source: v1.0 DOCX Section 4. Each system built in JobPilot maps directly to canonical SDE interview domains. Phase 2 additions are noted.

Domain	What You Built	Interview Application
System Design	Multi-agent LangGraph with state persistence, HITL checkpoints, cascading fallback	"Design a job matching system" — you built one. Agent decomposition, state machines, HITL, scaling.

Domain	What You Built	Interview Application
API Design	CascadingLLM with retry + circuit breaker; Lever/Greenhouse/Apollo integrations	"Design a gateway with fallback" — circuit breakers, timeouts, idempotency patterns.
Data Modelling	25-column tracker, pgvector embeddings, resume version metadata	"Design a schema for X" — normalisation, vector indexes, JSON vs relational trade-offs.
Distributed Sys.	Concurrent agents, APScheduler coordination, Docker multi-service	"Handle concurrent writes" — transaction isolation, locking, idempotent operations.
ML/AI Integration	RAG pipeline; LLM-as-judge, multi-agent tournament (Phase 2)	"Integrate LLMs into a product" — prompt eng, embeddings, cost optimisation, evaluation.
Monitoring	Prometheus + Grafana with funnel metrics (Phase 2); LangSmith tracing	"Monitor production services" — concrete metrics, thresholds, dashboards you built.
Security	OAuth 2.0 (Gmail), no raw email storage, token encryption	"Handle credentials" — OAuth flow, refresh tokens, encrypted storage, least privilege.
Testing	Unit per agent, integration pairs, load test (50 concurrent), 24h soak	"Test a complex system" — test pyramid, soak testing, LLM output validation.

Table 9. SDE interview competence map.

12. Monthly Operating Cost — Phase 1

Component	Cost / Month	Notes
LLM Tier 1 — DeepSeek V3 / Qwen 2.5	$0	50M tokens/day free tier. Covers all scoring, extraction, and outreach.
LLM Premium — Claude Sonnet 4.6	$1.50–3	~50–100 resume rewriting calls/month at $3 in / $15 out per 1M tokens.
LLM Tier 2 — Gemini Flash / GPT-4o-mini	$0–1	Overflow only. Gemini free tier is generous.
Ollama — Llama 3.2 3B (local GTX 1650)	$0	Terminal fallback only. Q4_K_M ~2GB, fits in 4GB VRAM.
Apollo.io (recruiter enrichment)	$0	50 free credits/month.
Telegram + Gmail API	$0	Free.
PostgreSQL + pgvector (home laptop)	$0	Already running on PAIOS infrastructure.
VPS hosting (optional, Hetzner CX22)	$0–4.50	Can run on home laptop instead.
Total Phase 1	$1.50–8.50	Typically $2–4/month at moderate job search volume.

Table 10. Monthly operating cost reference for Phase 1.

A. Appendix A — Phase 2: JobPilot Weekends 6–9

Phase 2 begins only after Phase 1 is running stably and has accumulated at least 20 real applications with known outcomes. All server infrastructure — PostgreSQL, FastAPI, APScheduler, and the Telegram bot — is reused without modification. Estimated additional investment: 80–100 hours.

> Prerequisite: The Adaptive Resume Engine requires ≥10 applications with known outcomes before activation. Email Intelligence requires real recruiter replies to build a classification baseline. Begin Phase 2 only after Phase 1 data accumulates.

A.1 Phase 2 Weekend Build Plan

Weekend	Agent(s)	Key Deliverable	Est. Hours
6	Email Intelligence Agent	Gmail OAuth + incremental sync every 5 min. Two-stage classification: regex pre-filter → cascading LLM. Seven categories. 0.85 confidence threshold routes ambiguous to Telegram. Acceptance signals feed the Adaptive Resume Engine.	16–20h
7	Follow-up Agent + ARE foundation	Follow-up: daily 10:00 IST scan, Day 7 and Day 14 drafts, mandatory Telegram approval, plain text only. ARE foundation: three analyst agents in parallel (Keyword, Phrasing, Structure) plus Judge agent with composite rubric (keyword 30%, phrasing 25%, structure 20%, ATS 15%, integrity 10%).	18–22h
8	ARE complete + MCP Servers	Mediator agent for deliberation when top-two scores fall within 5 points. Safety guardrails: no-fabrication diff-check, auto-revert on <10% acceptance for 3 cycles, max 30% bullets modified per cycle, /pin to freeze a version. Three MCP servers: jobpilot-scout, jobpilot-resume, jobpilot-outreach. Full 9-agent LangGraph integration.	16–20h
9	Analytics + Grafana + Final Deploy	Prometheus /metrics endpoint: 15+ counters, gauges, and histograms. Grafana 6-row dashboard: funnel, daily bar chart, response rate with resume version overlay, LLM cascade performance, outreach A/B comparison, ARE health and version timeline. Four alerting rules. Docker Compose final stack. 24h soak test. Tag v1.0 release.	14–18h

Table A1. Phase 2 weekend build plan.

A.2 Adaptive Resume Engine (ARE) — Architecture

Every 12 hours (06:00 and 18:00 IST), three analyst agents run in parallel and independently produce candidate base resumes:

- **Keyword Analyst.** Per-keyword success rates from Email Intelligence outcomes. Identifies which JD terms correlate with interview invites.
- **Phrasing Analyst.** Action verb and sentence structure correlation with application outcomes. Harvard verb effectiveness measured against real data.

- **Structure Analyst.** Bullet ordering, section emphasis, and skills prominence — optimised for ATS scores and recruiter attention patterns.

A Judge agent scores all three candidates against a composite rubric: keyword coverage (30%), phrasing quality (25%), structural coherence (20%), ATS compatibility (15%), factual integrity (10%). If the top two scores fall within 5 points, a Mediator agent synthesises a merged version. Full git-like version history is maintained with metadata, changelogs, and weekly diff reports via Telegram.

Safety guardrails — non-negotiable: Diff-check against original resume (no fabrication introduced). Auto-revert if acceptance rate drops below 10% for 3 consecutive cycles. Maximum 30% of bullets modified per cycle. Minimum 10 applications with known outcomes before first activation. /pin command freezes any version permanently.

A.3 Phase 2 Learning Requirements

Phase	New Learning	Hours	Key Resources
Phase 6 — Eval / Integrations	Gmail API incremental sync (historyId), OAuth 2.0 refresh flow, LLM-as-judge patterns, RAGAS, LangSmith tracing	10–12h	Gmail Quickstart Langfuse RAGAS
Phase 7 — Deploy / Monitor	Prometheus data model (counter/gauge/histogram), PromQL fundamentals, Grafana dashboard provisioning	10–12h	Prometheus Grafana prometheus_client

Table A2. Phase 2 learning requirements.

B. Appendix B — Project Directory Structure

Target state at end of Week 8 (Phase 1). Phase 2 directories (email_intelligence/, follow_up/, adaptive_resume/, monitoring/, mcp_servers/) are not shown.

```
jobpilot/
├── .env # All secrets — never commit
├── .env.example
├── docker-compose.yml # App + PostgreSQL + pgvector
├── README.md
├── config.yaml
├── core/
│ ├── cascade_llm.py # 4-tier fallback LLM client
│ ├── telegram_client.py # Telegram Bot + HITL helpers
│ ├── database.py # SQLAlchemy engine + schema init
│ └── models.py
├── agents/
│ ├── scout/
│ │ ├── scraper.py # Lever, Greenhouse, Naukri, LinkedIn, RSS
│ │ ├── scorer.py # LLM + pgvector hybrid relevance scoring
│ │ └── scheduler.py # APScheduler tiered frequency
│ ├── resume/
```

```
│ │ ├── latex_parser.py # .tex → structured intermediate
│ │ ├── rewriter.py # Bullet rewriting (Sonnet, no-fabrication)
│ │ ├── compiler.py # pdflatex + auto error recovery
│ │ └── ats_scorer.py
│ ├── application/
│ │ ├── lever.py
│ │ ├── email_submit.py # Gmail API + MIME
│ │ └── playwright_fill.py # Form auto-population fallback
│ ├── tracker/
│ │ ├── tracker.py # openpyxl 25-column tracker
│ │ └── daily_summary.py # 21:00 IST Telegram summary
│ └── outreach/
│ ├── apollo_finder.py # Apollo.io + Playwright fallback
│ └── message_gen.py # Dual-variant message generation
├── orchestration/
│ └── graph.py # LangGraph StateGraph (5 agents)
├── data/
│ ├── harvard_verbs.yaml # 200+ action verbs by category
│ ├── applicant_profile.yaml # Form auto-fill profile
│ └── resume_versions/ # LaTeX source history + changelogs
└── tests/
├── test_cascade_llm.py
├── test_scout.py
├── test_resume_agent.py
├── test_application_agent.py
└── test_outreach_agent.py
```

C. Appendix C — LLM Routing Decision Matrix

Task	Primary	Fallback	Justification
JD relevance scoring	DeepSeek V3 (free)	Qwen 2.5 → Gemini Flash	High volume, low stakes. Free tier sufficient.
JD keyword extraction	DeepSeek V3 (free)	Qwen 2.5	Structured JSON output. Free tier handles well.
Resume bullet rewriting	Claude Sonnet 4.6	Sonnet only — no fallback	Quality-critical. No-fabrication. No free-tier substitute.
ATS score estimation	DeepSeek V3 (free)	Qwen 2.5	Pattern matching. Free tier adequate.
Outreach message generation	DeepSeek V3 (free)	Qwen 2.5 → Gemini Flash	Medium quality bar. Dual-variant at free tier affordable.
Email classification (Phase 2)	DeepSeek V3 (free)	Qwen 2.5	High volume (5-min sync). Regex pre-filter reduces LLM calls.

Task	Primary	Fallback	Justification
ARE Keyword/Phrasing/Structure (P2)	DeepSeek V3 (free)	Qwen 2.5	Three parallel analysts per 12h cycle. Free tier handles volume.
ARE Judge agent (Phase 2)	Gemini 2.0 Flash	GPT-4o-mini	Quality-critical scoring. Small premium acceptable.
ARE Mediator / Narrative (Phase 2)	Claude Sonnet 4.6	Sonnet only — no fallback	Final synthesis affects hiring outcomes directly.
Terminal fallback — all tasks	Ollama Llama 3.2 3B	—	GTX 1650 4GB VRAM. Q4_K_M fits. $0. Used only when all APIs down.

Table C1. LLM routing decision matrix. Sonnet 4.6 has no fallback for quality-critical tasks.

Document generated: 01 April 2026 — 23:00 IST